

Last modified on

Sports Data Soccer API - Match Possession Value (MA30)

Get Possession Values for all players and teams for a particular match that's queried for.

Overview

The Soccer **Match Possession Value feed** provides all the traditional data such as match information, competition information and team details of all the players and teams part of this match, clubbed with new possession value like `totalPossessionValue`, `totalPlusPV`, `totalMinusPV`, `totalPlusPass`, `totalMinusPass`, `totalPlusCross`, `totalMinusCross`, `totalPlusTakeOns`, `totalMinusTakeOn`, `totalPlusSetPlaysTaken`, `totalMinusSetPlayTaken`, `totalPlusSetPlaysDrawn`, `totalMinusSetPlayConceded`, `totalPlusRegains` and `totalMinusLosingPossession`.

For more details refer: <https://www.theanalyst.com/eu/2021/03/what-is-possession-value/>

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our **FAQs**. We also recommend that you take a look at our **API Overview and Authentication Guide**. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY
	Once per minute in case a match that's queried for is "live".

Match Possession Value (MA30) Guide

USAGE	FEED
GET match possession values by fixture UUID (path parameter method).	<code>/soccerdata/matchpossessionvalues/{outletAuth_fmt=xml&_rt=b</code>
GET match possession values by fixture.	<code>/soccerdata/matchpossessionvalues/{outletAuth&_fmt=xml&_rt=b</code>

If your request URL is as follows - that is, it does not contain a fixture UUID the feed returns an error:
 /soccerdata/matchpossessionvalues/{outletAuthKey}

Query Parameters And Example Request Parameters

This feed also makes use of the **Global query parameters**, and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you must query the `/matchpossessionvalues` feed by a specific fixture (match) UUID and a **valid locale value**.

Legacy IDs: You can use the `fx` parameter to filter by legacy IDs - only one ID is supported (you cannot pass multiple IDs in a comma-separated list).

How to use query parameters and make successful requests:

In our guides, we use HTTPS syntax for our example `GET` requests (rather than `cURL`, for example). All URLs are case-sensitive - this means that URI strings must contain the correct case and operators. You **MUST** format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the main guide and **API Overview and Authorisation guide**.

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` (Outlet Authentication Key), query parameters for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format), entity query parameter and UUID.
- Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or - if a parameter supports it - `,` (multiple values, 'OR') or `!` ('NOT').
- Use the required `_` (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces `{ }` from your calls (we include these as an example only).
- If using the JSONP format (`_fmt=jsonp`) you **MUST** use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
<code>{fixtureUuid}</code> Fixture UUID	GET Match Possession values by the specified fixture UUID (path parameter method). Pass the fixture UUID in the request URL directly as the <code>{assetId}</code> (asset/resource ID). <pre>GET https://api.performfeeds.com/soccerdata/matchpossessionvalues/{outletAuthKey}/{fixtureUuid}?_fmt={dataFormat}&_rt={mode}</pre>
<code>fx</code> (Opta 'legacy' ID only)	GET Match Possession values by the specified Opta legacy 'fx' ID (query parameter method). Pass the match ID to the <code>fx</code> parameter. <pre>GET https://api.performfeeds.com/soccerdata/matchpossessionvalues/{outletAuthKey}?fx=urn:perform:opta:fixture:optamatchid&_fmt={dataFormat}&_rt={mode}</pre>

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- [XML](#)
- [JSON](#)

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

▶<matchPossessionValues> (XML only)

▶<matchInfo>

▶<description>

▶<sport>

▶<ruleset>

▶<competition>

▶<country>

▶<tournamentCalendar>

▶<stage>

▶<series>

▶<contestants>

▶<contestant>

▶<country>

▶<venue>

▶<possessionValue>

▶<contestant>

▶<pvCategories>

▶<lineUp>

▶<player>

▶<pvCategories>